Subset Sum

Reading: Erickson, §3.8, G/T §12.6 (related)

Problem Statement: Given a set S of n positive integers, as well as a positive integer T, determine if there is a subset of S that sums to exactly T.

Example 1: $S = \{2, 3, 4\}$, $T = 6$, the answer is "yes"

Example 2: $S = \{2, 3, 5\}$, $T = 6$, the answer is "no"

As with all other dynamic programming algorithms, we are going to start with a recursive case and transform it from there. Remember, *dynamic programming is about smart recursion.*

- Find a recursive algorithm to determine if a subset of the first n values in the input adds up to T.

- Finish the process to make this a dynamic programming algorithm, including outputting the subset of the items for the case when the answer is "yes." For example, your output on example one (above) should be "yes, 2, 4" while your output for example two should be "no."

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

What is the running time of the dynamic programming algorithm you gave for Subset Sum above?

- Suppose we double the size of S, but leave T alone. Will your algorithm scale well?

- Suppose we double the number of bits available to represent T, doubling its size, but leave S alone. Will your algorithm scale well?

One of the key terms here is *pseudo-polynomial*.

Candy at Office Hours

When classes were much smaller, I had a candy jar in my office. This question came from that era. Students would come to office hours not only seeking knowledge, but also because I give them candy; each student gets one piece of candy for each office hours attended. I have a candy jar in my office that can hold C pieces of candy. At any point, I can ask the department to add candy to the jar; they'll charge me a fixed price $\$P$ for doing so, regardless of how many pieces of candy I ask them for. They also don't like letting candy stay in the jar, getting stale; to discourage me from letting that happen, they charge me $\$d$ for each leftover piece after every office hour.

For the next quarter, I have n office hours planned; at the ith office hour, I project that s_i students will show up, each of whom will get one piece of candy. For this problem, assume that the projected attendance at office hours is accurate. At the start of the quarter, my candy jar is empty, and I aim to have it empty at the end of the semester.

While it is possible to solve this problem in time polynomial only in n, if I were collecting this for credit, full credit would be given for a correct solution with running time $\mathcal{O}(nC)$ (assuming the other required parts of the problem are fulfilled)

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Offline Optimal Binary Search Trees

Reading: Erickson §3.9, G/T 12.1 (related)

In ICS 46, you saw unbalanced binary search trees. These had $\mathcal{O}(\log n)$ lookup time under some conditions, but $\mathcal{O}(n)$ lookup time in the worst case. You then saw balanced binary search trees: there are structures such as AVL, Red/Black, WAVL, and Crumple Trees that make the "promise" that any given lookup in the tree would take $\mathcal{O}(\log n)$ time. We can't reasonably expect a better worst case, and these are great data structures for the case when elements can be added to the tree arbitrarily and we don't know how often (or even if) we will look up any given element.

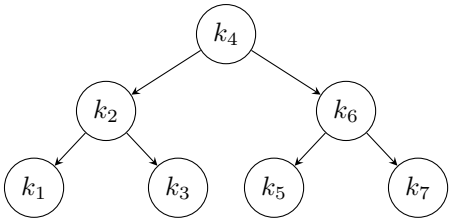
Suppose that before building a binary search tree, we knew exactly which elements were going to be in the tree. If we're likely to look up any given one with equal probability, or if we don't know the likelihood of looking up any given element, we can balance the tree by placing the median element at the root and recursively building trees in this fashion for the left- and right- subtrees.

But what if we also knew the probability that we'd look up any given element once the tree was built? This might not produce an optimal binary search tree in terms of the expected value of the lookup. Suppose we have n keys, $k_1 \dots k_n$, with probabilities $p_1 \dots p_n$ that we will look up the given elements; each probability p_i is positive, and the sum of these is 1.

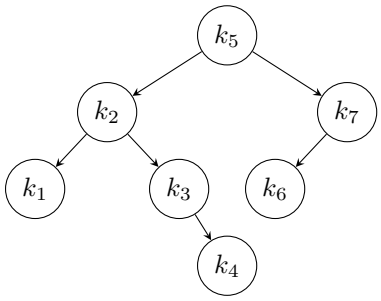
Here's an example with $n = 7$ keys:

i	1	2	3	4	5	6	7
p_i	.13	.21	.11	.01	.22	.08	.24

Here are two possible binary search trees with those keys:



This tree is balanced



This one is less balanced

What is the *expected* lookup cost (in terms of nodes examined) for each of these trees?

Problem Statement: We are given n probabilities, $p_1 \dots p_n$; p_i represents the probability of looking up the ith smallest key once the tree is built. Our goal is to build a binary search tree with the smallest expected lookup cost.
Note that our output must be a *binary search tree*; we cannot reorder the elements.

Check for understanding: we have computed d_i, the depth within the tree of each node. The root has $d_i = 1$, its children have $d_i = 2$, and so on. What is the expected lookup cost of this tree?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

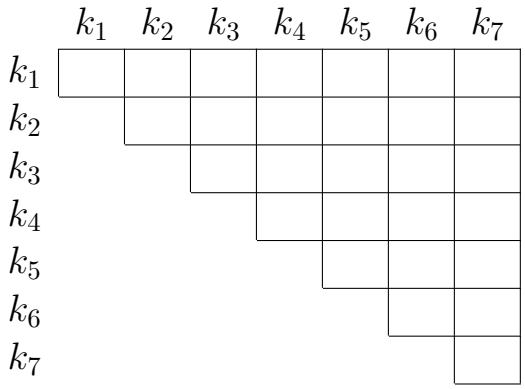
Creating the Dynamic Programming Algorithm

Let's compute Tree(i, j), which is going to be the *cost* of the optimal binary search tree consisting of keys i through j (inclusive). If this is called with $j < i$, we consider this a null tree and return 0 (treat this as a base case).

- Which key(s) can be the root of a binary search tree consisting of keys i through j?

- Suppose key r is the root. What is the cost of the search tree, rooted at r, consisting of keys i through j? You may assume that the left- and right- subtrees of r are constructed optimally.

Let's use that information to create a dynamic programming algorithm. When we're done, we will use that information to construct the tree itself.



The dynamic programming table before any are filled in. Any spaces that will remain unused are not pictured. I recommend filling in a few by hand (use a calculator as appropriate) to see the order in which they are filled in, and why that matters. See what happens, for example, if you tried to fill in the first row (in its entirety) first.

	k_1	k_2	k_3	k_4	k_5	k_6	k_7
k_1	0.13	0.47	0.69	0.72	1.28	1.52	2.12
k_2		0.21	0.43	0.46	1	1.17	1.73
k_3			0.11	0.13	0.47	0.63	1.19
k_4				0.01	0.24	0.4	0.95
k_5					0.22	0.38	0.92
k_6						0.08	0.4
k_7							0.24

The dynamic programming table after the program finishes with the sample input. Use this to check your work along the way if you take the previous advice.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

- Why can't we use the following to fill in the table of costs?

  for $i \leftarrow 1 \ldots n$ **do**
    for $j \leftarrow i \ldots n$ **do**
      Fill in Tree$[i, j]$

- In which order do we fill in table elements?

  for $i \leftarrow 1 \ldots n$ **do**
    Tree$[i, i-1] \leftarrow 0$
    Tree$[i, i] \leftarrow p_i$

Question 1. How do we get the tree itself from the table? Here is the table with the *costs* of the tree (this is the same table as on the previous digital page).

	k_1	k_2	k_3	k_4	k_5	k_6	k_7
k_1	0.13	0.47	0.69	0.72	1.28	1.52	2.12
k_2		0.21	0.43	0.46	1	1.17	1.73
k_3			0.11	0.13	0.47	0.63	1.19
k_4				0.01	0.24	0.4	0.95
k_5					0.22	0.38	0.92
k_6						0.08	0.4
k_7							0.24

Question 2. Write a function to, given the above table, construct the optimal binary search tree.

You should be able to reconstruct the tree from the table with the additions we make.

A good reinforcement activity would be to write a function that, given the full table, reconstructs the optimal tree.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Matrix Chain Multiplication

Problem Statement: Suppose we wish to compute the product of n matrices: $A_1A_2 \dots A_n$. Because matrix multiplication is associative, any parenthsization of this expression will yield the same result. We say a product of matrices is **fully parenthesized** if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses. For example, if $n = 4$, we can fully parenthesize the product in five ways:

- $(A_1(A_2(A_3A_4)))$
- $(A_1((A_2A_3)A_4))$
- $((A_1A_2)(A_3A_4))$
- $((A_1(A_2A_3))A_4)$
- $(((A_1A_2)A_3)A_4)$

While each of these produces the same *resulting product*, they do not require the same amount of work to produce.

The following is a quick reminder of how we multiply two matrices. First, it must be the case that the number of columns in A is equal to the number of rows in B. If A is $p \times q$ and B is $q \times r$, then the resulting matrix will be $p \times r$ and will require pqr scalar multiplications to produce.

Example: Suppose we wish to multiply three matrices $A_1A_2A_3$. The dimensions, respectively, are $10 \times 100, 100 \times 5, 5 \times 50$. Would it be better to multiply this as $((A_1A_2), A_3)$ or $(A_1(A_2A_3))$? That is, which of those options will require fewer *scalar* multiplications?

For this problem, we are looking into deciding which order to multiply the matrices; the actual multiplications of them is not being done here.

Input: If we are to multiply n matrices, we need only the relative dimensions. We know that A_i's columns must equal the number of rows in A_{i+1} for our input to be valid, so we need only to know the number of rows in each matrix, plus the number of columns in A_n. The rows in each matrix are equal to the number of columns in the previous matrix.

Thus, our input is just a list of $n + 1$ positive integers: $d_0d_1 \dots d_n$, where matrix i is of dimension $d_{i-1} \times d_i$ for i in the range $1 \dots n$.

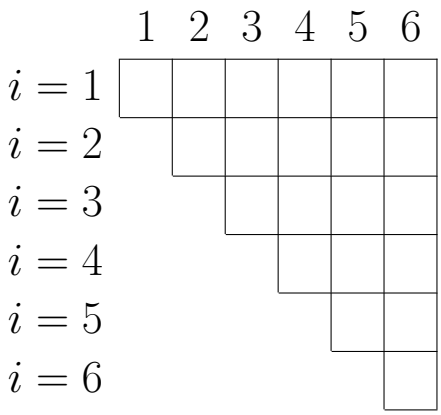
© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Creating the Dynamic Programming Algorithm

Let's compute MatrixChain(i, j), which is going to be the *cost* (in scalar multiplications) of the optimal matrix chain multiplication to produce the product of matrices i through j (inclusive). If this is called with $j = i$, we consider this an existing matrix and return 0 (treat this as a base case).

- Suppose matrix k is the last matrix in the "left split." What is the cost of the total multiplication, consisting of producing the matrix product for the left split, the right split, and the product of the two splits?

Let's use that information to create a dynamic programming algorithm. When we're done, we will use that information to construct the parenthesization itself.



The dynamic programming table before any are filled in. Any spaces that will remain unused are not pictured.

	1	2	3	4	5	6
$i = 1$	0	15,750 1	7,875 1	9,375 3	11,875 3	15,125 3
$i = 2$		0	2,625 2	4,375 3	7,125 3	10,500 3
$i = 3$			0	750 3	2500 3	5,375 3
$i = 4$				0 0	1,000 4	3,500 5
$i = 5$					0	5,000 5
$i = 6$						0

The dynamic programming table after the program finishes with an example input of
$d = \{30, 35, 15, 5, 10, 20, 25\}$

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.